Push Notifications - Standard Operating Procedure (SOP)

Generic Implementation Guide for Any Project

Document Version: 2.0

Last Updated: December 2024

Purpose: Universal SOP for implementing Firebase Cloud Messaging push notifications in any web or mobile application

---

Table of Contents

Overview

What This SOP Covers

This Standard Operating Procedure provides a complete, step-by-step guide for implementing Firebase Cloud Messaging (FCM) push notifications in any web or mobile application. It covers:

- □ Web push notifications (all modern browsers)
- □ Mobile push notifications (Android & iOS)
- □ Multi-device support
- □ Background and foreground notifications
- □ Notification click handling and navigation

Technology Stack

- **Firebase Cloud Messaging (FCM)** - Google's push notification service
- **Firebase Admin SDK** - Backend notification sending
- **Firebase JavaScript SDK** - Frontend token management
- **Service Workers** - Background notification handling (Web)
- **Native FCM SDK** - Mobile app integration

Use Cases

This implementation is suitable for:

- Web applications (React, Vue, Angular, vanilla JS)
- Mobile applications (React Native, Flutter, native iOS/Android)
- Full-stack applications (Node.js, Python, Java, etc.)
- Multi-platform applications (web + mobile)

Prerequisites

Required Accounts & Services

1. **Google Account** - For Firebase Console access
2. **Firebase Project** - Create or use existing project
3. **Backend Server** - Node.js, Python, Java, or any server framework
4. **Frontend Application** - Web app or mobile app
5. **Database** - Any database (MongoDB, PostgreSQL, MySQL, etc.)

Required Knowledge

- Basic understanding of REST APIs
- Familiarity with your backend framework
- Basic understanding of frontend JavaScript
- Understanding of environment variables

System Requirements

- **Backend:** Node.js 14+ (if using Node.js) or equivalent runtime
- **Frontend:** Modern browser with service worker support
- **Mobile:** Android 4.1+ or iOS 10+
- **HTTPS:** Required for production (localhost allowed for development)

Firebase Console Setup

Step 1: Create Firebase Project

1. Go to Firebase Console (https://console.firebase.google.com)
2. Sign in with your Google account
3. Click **"Add Project"** or select existing project
4. Enter project name (e.g., `my-app-notifications`)
5. **Note your Project ID** - You'll need this later
6. Follow setup wizard (Google Analytics optional)

Step 2: Register Web App

1. In Firebase Console, click the **web icon** (`</>`)
2. Register your app with a nickname (e.g., "My Web App")
3. **Copy the Firebase configuration** - You'll need these values:

```
{
  apiKey: "AIza...",
  authDomain: "your-project.firebaseapp.com",
  projectId: "your-project-id",
  storageBucket: "your-project.firebasestorage.app",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abc123",
  measurementId: "G-XXXXXXXXXX"
}
```

4. **Save this configuration** - You'll use it in frontend

Step 3: Enable Cloud Messaging

1. In Firebase Console, go to **Project Settings** (gear icon)

2. Click **Cloud Messaging** tab
3. Scroll to **Web Push certificates** section
4. Click **Generate key pair** (if not already generated)
5. **Copy the VAPID key** - Save this securely
    - Example format: `BAXnzclIUpol3ExXQV8JokW7plpWqSJhLIFrXlnNHueIylJFuC3TQ17wWRIspB4IOmi-NffJuWq2mz9C6sC1YlQ`
6. **Important:** This VAPID key is needed for frontend

Step 4: Generate Service Account Key (Backend)

1. In Firebase Console, go to **Project Settings** > **Service Accounts** tab
2. Click **Generate new private key**
3. Confirm by clicking **Generate key**
4. A JSON file will download - **SAVE THIS SECURELY**
5. File name format: `your-project-firebase-adminsdk-xxxxx-xxxxx.json`
6. **Important:** This file gives admin access - keep it secure!

Step 5: Verify Cloud Messaging API (Optional)

Note: Usually auto-enabled, but verify if needed:

1. Go to Google Cloud Console (https://console.cloud.google.com)
2. Select your Firebase project
3. Navigate to **APIs & Services** > **Library**
4. Search for "Firebase Cloud Messaging API"
5. Ensure it shows "Enabled" - if not, click **Enable**

Backend Implementation

Step 1: Install Firebase Admin SDK

For Node.js:

```
npm install firebase-admin
```

For Python:

```
pip install firebase-admin
```

For Java:

Add to `pom.xml`:

```
<dependency>
    <groupId>com.google.firebase</groupId>
    <artifactId>firebase-admin</artifactId>
    <version>9.2.0</version>
</dependency>
```

Step 2: Initialize Firebase Admin

Node.js Example:

```
// services/firebaseAdmin.js
const admin = require('firebase-admin');
const path = require('path');

// Option 1: Service Account File
const serviceAccount = require('../config/firebase-service-account.json');

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

// Function to send notification
async function sendPushNotification(tokens, payload) {
  try {
    const message = {
      notification: {
        title: payload.title,
        body: payload.body,
      },
      data: payload.data || {},
      tokens: tokens, // Array of FCM tokens
    };

    const response = await admin.messaging().sendEachForMulticast(message);
    console.log(`Successfully sent: ${response.successCount} messages`);
    console.log(`Failed: ${response.failureCount} messages`);

    return response;
  } catch (error) {
    console.error('Error sending message:', error);
    throw error;
  }
}

module.exports = { sendPushNotification };
```

Python Example:

```
# services/firebase_admin.py
import firebase_admin
from firebase_admin import credentials, messaging

# Initialize Firebase Admin
cred = credentials.Certificate('config/firebase-service-account.json')
firebase_admin.initialize_app(cred)

def send_push_notification(tokens, payload):
    """Send push notification to multiple tokens"""
    message = messaging.MulticastMessage(
        notification=messaging.Notification(
            title=payload['title'],
            body=payload['body']
        ),
        data=payload.get('data', {}),
        tokens=tokens
    )

    response = messaging.send_each_for_multicast(message)
    print(f"Successfully sent: {response.success_count} messages")
    print(f"Failed: {response.failure_count} messages")

    return response
```

Step 3: Environment Variables

Create `.env` file:

```
# Option 1: Service Account File Path
FIREBASE_SERVICE_ACCOUNT_PATH=./config/firebase-service-account.json

# Option 2: Full JSON as Environment Variable (Production - Recommended)
# FIREBASE_CONFIG={"type":"service_account","project_id":"your-project-id",...}
```

Step 4: Place Service Account File

1. Place downloaded service account JSON file in secure location
2. Recommended: `backend/config/firebase-service-account.json`
3. **Important:** Add to `.gitignore`:

```
config/firebase-service-account.json
*.json
```

Step 5: Create FCM Token Endpoints

Node.js/Express Example:

```
// routes/fcmTokenRoutes.js
const express = require('express');
const router = express.Router();
const { authenticate } = require('../middlewares/auth'); // Your auth middleware

// Save FCM Token
router.post('/save', authenticate, async (req, res) => {
  try {
    const { token, platform = 'web' } = req.body;
    const userId = req.user.id; // From auth middleware

    // Get user model (adjust based on your database)
    const User = require('../models/User');
    const user = await User.findById(userId);

    // Add token to array (web or mobile)
    if (platform === 'web') {
      if (!user.fcmTokens) user.fcmTokens = [];
      if (!user.fcmTokens.includes(token)) {
        user.fcmTokens.push(token);
        // Limit to 10 tokens
        if (user.fcmTokens.length > 10) {
          user.fcmTokens = user.fcmTokens.slice(-10);
        }
      }
    } else if (platform === 'mobile') {
      if (!user.fcmTokenMobile) user.fcmTokenMobile = [];
      if (!user.fcmTokenMobile.includes(token)) {
        user.fcmTokenMobile.push(token);
        if (user.fcmTokenMobile.length > 10) {
          user.fcmTokenMobile = user.fcmTokenMobile.slice(-10);
        }
      }
    }

    await user.save();

    res.json({ success: true, message: 'FCM token saved' });
  } catch (error) {
    console.error('Error saving FCM token:', error);
    res.status(500).json({ error: 'Failed to save token' });
  }
});

// Remove FCM Token
router.delete('/remove', authenticate, async (req, res) => {
  try {
    const { token, platform = 'web' } = req.body;
    const userId = req.user.id;

    const User = require('../models/User');
    const user = await User.findById(userId);

    if (platform === 'web' && user.fcmTokens) {
      user.fcmTokens = user.fcmTokens.filter(t => t !== token);
    } else if (platform === 'mobile' && user.fcmTokenMobile) {
      user.fcmTokenMobile = user.fcmTokenMobile.filter(t => t !== token);
    }
```

```
    await user.save();
    res.json({ success: true, message: 'FCM token removed' });
  } catch (error) {
    console.error('Error removing FCM token:', error);
    res.status(500).json({ error: 'Failed to remove token' });
  }
});

module.exports = router;
```

Step 6: Update Database Models

Add FCM token fields to your user model:

MongoDB/Mongoose Example:

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  // ... your existing fields
  email: String,
  name: String,

  // FCM Tokens
  fcmTokens: {
    type: [String],
    default: []
  },
  fcmTokenMobile: {
    type: [String],
    default: []
  }
});

module.exports = mongoose.model('User', userSchema);
```

SQL Example (PostgreSQL):

```
-- Add columns to users table
ALTER TABLE users
ADD COLUMN fcm_tokens TEXT[] DEFAULT '{}',
ADD COLUMN fcm_token_mobile TEXT[] DEFAULT '{}';
```

Step 7: Send Notification Function

Create helper function:

```
// utils/pushNotificationHelper.js
const { sendPushNotification } = require('../services/firebaseAdmin');
const User = require('../models/User');

async function sendNotificationToUser(userId, payload, includeMobile = true) {
  try {
    const user = await User.findById(userId);
    if (!user) {
      throw new Error('User not found');
    }

    // Collect tokens
    let tokens = [];
    if (user.fcmTokens && user.fcmTokens.length > 0) {
      tokens = [...tokens, ...user.fcmTokens];
    }
    if (includeMobile && user.fcmTokenMobile && user.fcmTokenMobile.length > 0) {
      tokens = [...tokens, ...user.fcmTokenMobile];
    }

    // Remove duplicates
    const uniqueTokens = [...new Set(tokens)];

    if (uniqueTokens.length === 0) {
      console.log('No FCM tokens found for user');
      return;
    }

    // Send notification
    await sendPushNotification(uniqueTokens, payload);
  } catch (error) {
    console.error('Error sending notification:', error);
    // Don't throw - notifications are non-critical
  }
}

module.exports = { sendNotificationToUser };
```

Step 8: Use in Controllers

Example: Send notification when action occurs

```
// controllers/taskController.js
const { sendNotificationToUser } = require('../utils/pushNotificationHelper');

async function createTask(req, res) {
  try {
    // ... your task creation logic
    const task = await Task.create({...});

    // Send notification to assigned user
    if (task.assignedTo) {
      await sendNotificationToUser(
        task.assignedTo,
        {
          title: 'New Task Assigned',
          body: `You have been assigned: ${task.title}`,
          data: {
            type: 'task',
            id: task._id.toString(),
            link: `/tasks/${task._id}`
          }
        }
      );
    }

    res.json({ success: true, task });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

Frontend Implementation

Step 1: Install Firebase SDK

For React/Vue/Angular:

```
npm install firebase
```

For Vanilla JavaScript:

```
<script src="https://www.gstatic.com/firebasejs/9.0.0/firebase-app.js"></script>
<script src="https://www.gstatic.com/firebasejs/9.0.0/firebase-messaging.js"></script>
```

Step 2: Initialize Firebase

Create `src/firebase.js` (or equivalent):

```
import { initializeApp } from 'firebase/app';
import { getMessaging, getToken, onMessage } from 'firebase/messaging';

const firebaseConfig = {
  apiKey: import.meta.env.VITE_FIREBASE_API_KEY,
  authDomain: import.meta.env.VITE_FIREBASE_AUTH_DOMAIN,
  projectId: import.meta.env.VITE_FIREBASE_PROJECT_ID,
  storageBucket: import.meta.env.VITE_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_FIREBASE_MESSAGING_SENDER_ID,
  appId: import.meta.env.VITE_FIREBASE_APP_ID,
  measurementId: import.meta.env.VITE_FIREBASE_MEASUREMENT_ID
};

const app = initializeApp(firebaseConfig);
const messaging = getMessaging(app);

export { messaging, getToken, onMessage };
```

Step 3: Environment Variables

Create `.env` file:

```
VITE_FIREBASE_API_KEY=your_api_key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=123456789
VITE_FIREBASE_APP_ID=1:123456789:web:abc123
VITE_FIREBASE_MEASUREMENT_ID=G-XXXXXXXXXX
VITE_FIREBASE_VAPID_KEY=your_vapid_key_here
```

Step 4: Create Service Worker

Create `public/firebase-messaging-sw.js`:

```
// Import Firebase scripts
importScripts('https://www.gstatic.com/firebasejs/9.0.0/firebase-app.js');
importScripts('https://www.gstatic.com/firebasejs/9.0.0/firebase-messaging.js');

// Firebase configuration (same as frontend)
const firebaseConfig = {
  apiKey: 'YOUR_API_KEY',
  authDomain: 'YOUR_AUTH_DOMAIN',
  projectId: 'YOUR_PROJECT_ID',
  storageBucket: 'YOUR_STORAGE_BUCKET',
  messagingSenderId: 'YOUR_SENDER_ID',
  appId: 'YOUR_APP_ID',
  measurementId: 'YOUR_MEASUREMENT_ID'
};

// Initialize Firebase
firebase.initializeApp(firebaseConfig);

// Get messaging instance
const messaging = firebase.messaging();

// Handle background messages
messaging.onBackgroundMessage((payload) => {
  console.log('[firebase-messaging-sw.js] Received background message', payload);

  const notificationTitle = payload.notification.title;
  const notificationOptions = {
    body: payload.notification.body,
    icon: payload.notification.icon || '/favicon.png',
    data: payload.data
  };

  self.registration.showNotification(notificationTitle, notificationOptions);
});

// Handle notification click
self.addEventListener('notificationclick', (event) => {
  event.notification.close();

  const data = event.notification.data;
  const urlToOpen = data?.link || '/';

  event.waitUntil(
    clients.matchAll({ type: 'window', includeUncontrolled: true }).then((clientList) => {
      // Check if app is already open
      for (const client of clientList) {
        if (client.url === urlToOpen && 'focus' in client) {
          return client.focus();
        }
      }
      // Open new window
      if (clients.openWindow) {
        return clients.openWindow(urlToOpen);
      }
    })
  );
});
```

Important: Replace placeholder values with actual Firebase config values.

Step 5: Create Push Notification Service

Create `src/services/pushNotificationService.js`:

```
import { messaging, getToken, onMessage } from '../firebase';

const VAPID_KEY = import.meta.env.VITE_FIREBASE_VAPID_KEY;

// Register service worker
async function registerServiceWorker() {
  if ('serviceWorker' in navigator) {
    try {
      const registration = await navigator.serviceWorker.register('/firebase-messaging-sw.js');
      console.log('□ Service Worker registered:', registration);
      return registration;
    } catch (error) {
      console.error('□ Service Worker registration failed:', error);
      throw error;
    }
  } else {
    throw new Error('Service Workers are not supported');
  }
}

// Request notification permission
async function requestNotificationPermission() {
  if ('Notification' in window) {
    const permission = await Notification.requestPermission();
    if (permission === 'granted') {
      console.log('□ Notification permission granted');
      return true;
    } else {
      console.log('□ Notification permission denied');
      return false;
    }
  }
  return false;
}

// Get FCM token
async function getFCMToken() {
  try {
    const registration = await registerServiceWorker();
    await registration.update(); // Update service worker

    const token = await getToken(messaging, {
      vapidKey: VAPID_KEY,
      serviceWorkerRegistration: registration
    });

    if (token) {
      console.log('□ FCM Token obtained:', token);
      return token;
    } else {
      console.log('□ No FCM token available');
      return null;
    }
  } catch (error) {
    console.error('□ Error getting FCM token:', error);
    throw error;
  }
}
```

```
// Register FCM token with backend
async function registerFCMToken(forceUpdate = false) {
  try {
    // Check if already registered
    const savedToken = localStorage.getItem('fcm_token_web');
    if (savedToken && !forceUpdate) {
      console.log('FCM token already registered');
      return savedToken;
    }

    // Request permission
    const hasPermission = await requestNotificationPermission();
    if (!hasPermission) {
      throw new Error('Notification permission not granted');
    }

    // Get token
    const token = await getFCMToken();
    if (!token) {
      throw new Error('Failed to get FCM token');
    }

    // Save to backend
    const authToken = localStorage.getItem('authToken'); // Your auth token key
    const response = await fetch('/api/fcm-tokens/save', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${authToken}`
      },
      body: JSON.stringify({
        token: token,
        platform: 'web'
      })
    });

    if (response.ok) {
      localStorage.setItem('fcm_token_web', token);
      console.log('□ FCM token registered with backend');
      return token;
    } else {
      throw new Error('Failed to register token with backend');
    }
  } catch (error) {
    console.error('□ Error registering FCM token:', error);
    throw error;
  }
}

// Setup foreground notification handler
function setupForegroundNotificationHandler(handler) {
  onMessage(messaging, (payload) => {
    console.log('□ Foreground message received:', payload);

    // Show notification
    if ('Notification' in window && Notification.permission === 'granted') {
      new Notification(payload.notification.title, {
```

```
        body: payload.notification.body,
        icon: payload.notification.icon || '/favicon.png',
        data: payload.data
      });
    }

    // Call custom handler
    if (handler) {
      handler(payload);
    }
  });
}

// Initialize push notifications
async function initializePushNotifications() {
  try {
    await registerServiceWorker();
    // Token will be registered on login
  } catch (error) {
    console.error('Error initializing push notifications:', error);
  }
}

export {
  initializePushNotifications,
  registerFCMToken,
  setupForegroundNotificationHandler,
  requestNotificationPermission
};
```

Step 6: Initialize in App

In your main app file (e.g., `App.jsx`, `main.js`, `App.vue`):

```
import { initializePushNotifications, setupForegroundNotificationHandler } from './services/pushNotificationService';
import { useEffect } from 'react'; // or Vue's onMounted, etc.

function App() {
  useEffect(() => {
    // Initialize on app load
    initializePushNotifications();

    // Setup foreground handler
    setupForegroundNotificationHandler((payload) => {
      console.log('Notification received:', payload);
      // Handle navigation or other actions
      if (payload.data?.link) {
        // Navigate to link
        window.location.href = payload.data.link;
      }
    });
  }, []);

  // ... rest of app
}
```

Step 7: Register Token on Login

In your login component:

```
import { registerFCMToken } from '../services/pushNotificationService';

async function handleLogin(email, password) {
  try {
    // ... your login logic
    const response = await login(email, password);

    // After successful login, register FCM token
    if (response.success) {
      await registerFCMToken(true); // forceUpdate = true
    }

    // ... rest of login flow
  } catch (error) {
    console.error('Login error:', error);
  }
}
```

Mobile App Integration

React Native

Step 1: Install dependencies

```
npm install @react-native-firebase/app @react-native-firebase/messaging
```

Step 2: Initialize Firebase

```
import messaging from '@react-native-firebase/messaging';

// Request permission
async function requestUserPermission() {
  const authStatus = await messaging().requestPermission();
  const enabled =
    authStatus === messaging.AuthorizationStatus.AUTHORIZED ||
    authStatus === messaging.AuthorizationStatus.PROVISIONAL;

  if (enabled) {
    console.log('Authorization status:', authStatus);
  }
}

// Get FCM token
async function getFCMToken() {
  const token = await messaging().getToken();
  console.log('FCM Token:', token);
  return token;
}

// Save to backend
async function saveTokenToBackend(token) {
  const authToken = await AsyncStorage.getItem('authToken');
  await fetch('YOUR_API_URL/api/fcm-tokens/mobile/save', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${authToken}`
    },
    body: JSON.stringify({
      token: token,
      platform: 'mobile'
    })
  });
}
```

Flutter

Step 1: Add dependencies

```
dependencies:
  firebase_core: ^3.0.0
  firebase_messaging: ^15.0.0
  flutter_local_notifications: ^17.0.0
```

Step 2: Initialize Firebase

```
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}

// Get FCM token
Future<String?> getFCMToken() async {
  String? token = await FirebaseMessaging.instance.getToken();
  print('FCM Token: $token');
  return token;
}

// Save to backend
Future<void> saveTokenToBackend(String token) async {
  final response = await http.post(
    Uri.parse('YOUR_API_URL/api/fcm-tokens/mobile/save'),
    headers: {
      'Content-Type': 'application/json',
      'Authorization': 'Bearer $authToken',
    },
    body: jsonEncode({
      'token': token,
      'platform': 'mobile',
    }),
  );
}
```

Native Android

Step 1: Add to `build.gradle`

```
dependencies {
    implementation platform('com.google.firebase:firebase-bom:32.7.0')
    implementation 'com.google.firebase:firebase-messaging'
}
```

Step 2: Get FCM token

```
FirebaseMessaging.getInstance().getToken()
    .addOnCompleteListener(new OnCompleteListener<String>() {
        @Override
        public void onComplete(@NonNull Task<String> task) {
            if (!task.isSuccessful()) {
                return;
            }
            String token = task.getResult();
            // Save to backend
            saveTokenToBackend(token);
        }
    });
```

Native iOS

Step 1: Add Firebase to project

1. Add `GoogleService-Info.plist` to Xcode project
2. Enable Push Notifications capability

Step 2: Get FCM token

```
Messaging.messaging().token { token, error in
  if let error = error {
    print("Error fetching FCM registration token: \(error)")
  } else if let token = token {
    print("FCM registration token: \(token)")
    // Save to backend
    saveTokenToBackend(token)
  }
}
```

Testing & Verification

Step 1: Test Service Worker Registration

1. Open browser DevTools (F12)
2. Go to **Application** tab > **Service Workers**
3. Verify `firebase-messaging-sw.js` is registered
4. Check for errors

Step 2: Test Permission Request

1. Open app in browser
2. Browser should prompt for notification permission
3. Click "Allow"
4. Check console for: □ `Notification permission granted`

Step 3: Test Token Generation

1. Check browser console for: □ `FCM Token obtained: ...`
2. Verify token is a long string
3. Check backend logs for token save confirmation

Step 4: Test Notification Sending

Create test endpoint in backend:

```
// routes/fcmTokenRoutes.js
router.post('/test', authenticate, async (req, res) => {
  try {
    const userId = req.user.id;
    const User = require('../models/User');
    const user = await User.findById(userId);

    const tokens = [...(user.fcmTokens || []), ...(user.fcmTokenMobile || [])];
    const uniqueTokens = [...new Set(tokens)];

    if (uniqueTokens.length === 0) {
      return res.json({ error: 'No FCM tokens found' });
    }

    await sendPushNotification(uniqueTokens, {
      title: 'Test Notification',
      body: 'This is a test notification',
      data: {
        type: 'test',
        link: '/'
      }
    });

    res.json({ success: true, message: 'Test notification sent' });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

Test from browser console:

```
fetch('/api/fcm-tokens/test', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${localStorage.getItem('authToken')}`
  }
})
.then(r => r.json())
.then(console.log);
```

Step 5: Test Foreground vs Background

Foreground:

1. Keep app tab active
2. Send notification
3. Check console for: □ `Foreground message received`

Background:

1. Minimize browser or switch tabs
2. Send notification
3. Browser notification should appear
4. Click notification → App should open

Troubleshooting

Issue 1: Service Worker Not Registering

Solutions:

- Ensure file is in `public/` folder (or equivalent)
- Check HTTPS (required for production, localhost OK for dev)
- Clear browser cache
- Check file path matches registration path

Issue 2: Token Not Generating

Solutions:

- Check VAPID key is correct
- Verify service worker is registered
- Check notification permission is granted
- Verify Firebase config is correct

Issue 3: Notifications Not Received

Solutions:

- Verify token is saved in database
- Check Firebase Admin is initialized
- Verify service account file is correct
- Check notification payload structure
- Test with background tab (browsers suppress foreground notifications)

Issue 4: Permission Denied

Solutions:

- Clear site data and refresh
- Check browser notification settings
- Manually allow notifications in browser settings

Issue 5: Backend Errors

Solutions:

- Verify service account file path
- Check environment variables
- Verify Firebase Admin initialization
- Check backend logs for specific errors

Production Deployment

Backend Deployment

1. **Service Account File:**
    - Upload to secure server location
    - Set file permissions: `chmod 600`
    - Use environment variable for path
    - Or use `FIREBASE_CONFIG` environment variable (recommended)
2. **Environment Variables:**

```
FIREBASE_SERVICE_ACCOUNT_PATH=/secure/path/to/service-account.json
# OR
FIREBASE_CONFIG={"type":"service_account",...}
```

3. **Security:**
   - Never commit service account file to git
   - Use environment variables in production
   - Restrict file permissions

Frontend Deployment

1. **Update Service Worker:**
   - Replace placeholder values in `firebase-messaging-sw.js`
   - Commit updated file
2. **Build:**

```
npm run build
```

3. **Verify:**
   - Service worker accessible at `/firebase-messaging-sw.js`
   - HTTPS enabled (required)
   - Firebase config correct

Mobile App Deployment

1. **Android:**
   - Ensure `google-services.json` is in `android/app/`
   - Build and deploy
2. **iOS:**
   - Ensure `GoogleService-Info.plist` is in project
   - Enable Push Notifications capability
   - Build and deploy

Best Practices

Security

1. **Never commit service account files to git**
2. **Use environment variables for sensitive data**
3. **Restrict file permissions (600)**
4. **Use HTTPS in production**
5. **Validate tokens on backend**

Performance

1. **Limit tokens per user (recommended: 10)**
2. **Remove invalid tokens automatically**
3. **Batch notifications when possible**
4. **Handle errors gracefully (don't break main flow)**

User Experience

1. **Request permission at appropriate time**
2. **Provide clear notification content**
3. **Include navigation links in notifications**
4. **Don't spam users with notifications**
5. **Handle notification clicks properly**

Code Quality

1. **Handle errors gracefully**
2. **Log important events**
3. **Use consistent payload structure**
4. **Document notification types**
5. **Test thoroughly before deployment**

Quick Reference

API Endpoints

```
POST /api/fcm-tokens/save          # Save web token
POST /api/fcm-tokens/mobile/save   # Save mobile token
DELETE /api/fcm-tokens/remove      # Remove token
POST /api/fcm-tokens/test          # Send test notification
```

Notification Payload

```
{
  title: "Title",           // Required
  body: "Message",          // Required
  data: {                   // Optional
    type: "task",
    id: "123",
    link: "/tasks/123"
  },
  icon: "/favicon.png"      // Optional
}
```

Environment Variables

Backend:

```
FIREBASE_SERVICE_ACCOUNT_PATH=./config/firebase-service-account.json
```

Frontend:

```
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_FIREBASE_MEASUREMENT_ID=...
VITE_FIREBASE_VAPID_KEY=...
```

Support & Resources

- **Firebase Documentation:** https://firebase.google.com/docs/cloud-messaging (https://firebase.google.com/docs/cloud-messaging)
- **Web Push Notifications:** https://web.dev/push-notifications-overview/ (https://web.dev/push-notifications-overview/)
- **Service Workers:** https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API (https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API)
- **FCM Best Practices:** https://firebase.google.com/docs/cloud-messaging/best-practices (https://firebase.google.com/docs/cloud-messaging/best-practices)

Document Version: 2.0

Last Updated: December 2024

Purpose: Universal SOP for FCM Push Notifications

Status: Production Ready □